

Handling relative mouse movement

To access relative mouse delta values, register for the `MouseDevice::MouseMoved` event as shown here.

C++

```
// register handler for relative mouse movement events
Windows::Devices::Input::MouseDevice::GetForCurrentView()->MouseMoved +=
    ref new TypedEventHandler<MouseDevice^, MouseEventArgs^>(this,
&MoveLookController::OnMouseMoved);
```

C++

```
void MoveLookController::OnMouseMoved(
    _In_ Windows::Devices::Input::MouseDevice^ mouseDevice,
    _In_ Windows::Devices::Input::MouseEventArgs^ args
)
{
    float2 pointerDelta;
    pointerDelta.x = static_cast<float>(args->MouseDelta.X);
    pointerDelta.y = static_cast<float>(args->MouseDelta.Y);

    float2 rotationDelta;
    rotationDelta = pointerDelta * ROTATION_GAIN;    // scale for control
    sensitivity

    // update our orientation based on the command
    m_pitch -= rotationDelta.y;                      // mouse y increases
    down, but pitch increases up
    m_yaw -= rotationDelta.x;                        // yaw defined as CCW
    around y-axis

    // limit pitch to straight up or straight down
    float limit = (float)(M_PI/2) - 0.01f;
    m_pitch = (float) __max( -limit, m_pitch );
    m_pitch = (float) __min( +limit, m_pitch );

    // keep longitude in useful range by wrapping
    if ( m_yaw > M_PI )
        m_yaw -= (float)M_PI*2;
    else if ( m_yaw < -M_PI )
        m_yaw += (float)M_PI*2;
}
```

The event handler in this code example, **OnMouseMoved**, renders the view based on the movements of the mouse. The position of the mouse pointer is passed to the handler as a [MouseEventArgs](#) object.

Skip over processing of absolute mouse data from the [CoreWindow::PointerMoved](#) event when your app changes to handling relative mouse movement values. However, only skip this input if the **CoreWindow::PointerMoved** event occurred as the result of mouse input (as opposed to touch input). The cursor is hidden by setting the [CoreWindow::PointerCursor](#) to **nullptr**.

Returning to absolute mouse movement

When the app exits the 3-D object or scene manipulation mode and no longer uses relative mouse movement (such as when it returns to a menu screen), return to normal processing of absolute mouse movement. At this time, stop reading relative mouse data, restart the processing of standard mouse (and pointer) events, and set [CoreWindow::PointerCursor](#) to non-null value.

Note

When your app is in the 3-D object/scene manipulation mode (processing relative mouse movements with the cursor off), the mouse cannot invoke edge UI such as the charms, back stack, or app bar. Therefore, it is important to provide a mechanism to exit this particular mode, such as the commonly used **Esc** key.

Related topics

- [Move-look controls for games](#)
- [Touch controls for games](#)

Touch controls for games

Article • 10/20/2022

Learn how to add basic touch controls to your Universal Windows Platform (UWP) C++ game with DirectX. We show you how to add touch-based controls to move a fixed-plane camera in a Direct3D environment, where dragging with a finger or stylus shifts the camera perspective.

You can incorporate these controls in games where you want the player to drag to scroll or pan over a 3D environment, such as a map or playfield. For example, in a strategy or puzzle game, you can use these controls to let the player view a game environment that is larger than the screen by panning left or right.

Note Our code also works with mouse-based panning controls. The pointer related events are abstracted by the Windows Runtime APIs, so they can handle either touch- or mouse-based pointer events.

Objectives

- Create a simple touch drag control for panning a fixed-plane camera in a DirectX game.

Set up the basic touch event infrastructure

First, we define our basic controller type, the **CameraPanController**, in this case. Here, we define a controller as an abstract idea, the set of behaviors the user can perform.

The **CameraPanController** class is a regularly refreshed collection of information about the camera controller state, and provides a way for our app to obtain that information from its update loop.

C++

```
using namespace Windows::UI::Core;
using namespace Windows::System;
using namespace Windows::Foundation;
using namespace Windows::Devices::Input;
#include <directxmath.h>

// Methods to get input from the UI pointers
```

```
ref class CameraPanController
{
}
```

Now, let's create a header that defines the state of the camera controller, and the basic methods and event handlers that implement the camera controller interactions.

C++

```
ref class CameraPanController
{
private:
    // Properties of the controller object
    DirectX::XMFLOAT3 m_position;           // the position of the
    camera

    // Properties of the camera pan control
    bool m_panInUse;
    uint32 m_panPointerID;
    DirectX::XMFLOAT2 m_panFirstDown;
    DirectX::XMFLOAT2 m_panPointerPosition;
    DirectX::XMFLOAT3 m_panCommand;

internal:
    // Accessor to set the position of the controller
    void SetPosition( _In_ DirectX::XMFLOAT3 pos );

    // Accessor to set the fixed "look point" of the controller
    DirectX::XMFLOAT3 get_FixedLookPoint();

    // Returns the position of the controller object
    DirectX::XMFLOAT3 get_Position();

public:

    // Methods to get input from the UI pointers
    void OnPointerPressed(
        _In_ Windows::UI::Core::CoreWindow^ sender,
        _In_ Windows::UI::Core::PointerEventArgs^ args
    );

    void OnPointerMoved(
        _In_ Windows::UI::Core::CoreWindow^ sender,
        _In_ Windows::UI::Core::PointerEventArgs^ args
    );

    void OnPointerReleased(
        _In_ Windows::UI::Core::CoreWindow^ sender,
        _In_ Windows::UI::Core::PointerEventArgs^ args
    );

    // Set up the Controls supported by this controller
    void Initialize( _In_ Windows::UI::Core::CoreWindow^ window );
```

```
void Update( Windows::UI::Core::CoreWindow ^window );  
  
}; // Class CameraPanController
```

The private fields contain the current state of the camera controller. Let's review them.

- **m_position** is the position of the camera in the scene space. In this example, the z-coordinate value is fixed at 0. We could use a `DirectX::XMFLOAT2` to represent this value, but for the purposes of this sample and future extensibility, we use a `DirectX::XMFLOAT3`. We pass this value through the **get_Position** property to the app itself so it can update the viewport accordingly.
- **m_panInUse** is a Boolean value that indicates whether a pan operation is active; or, more specifically, whether the player is touching the screen and moving the camera.
- **m_panPointerID** is a unique ID for the pointer. We won't use this in the sample, but it's a good practice to associate your controller state class with a specific pointer.
- **m_panFirstDown** is the point on the screen where the player first touched the screen or clicked the mouse during the camera pan action. We use this value later to set a dead zone to prevent jitter when the screen is touched, or if the mouse shakes a little.
- **m_panPointerPosition** is the point on the screen where the player has currently moved the pointer. We use it to determine what direction the player wanted to move by examining it relative to **m_panFirstDown**.
- **m_panCommand** is the final computed command for the camera controller: up, down, left, or right. Because we are working with a camera fixed to the x-y plane, this could be a `DirectX::XMFLOAT2` value instead.

We use these 3 event handlers to update the camera controller state info.

- **OnPointerPressed** is an event handler that our app calls when the players presses a finger onto the touch surface and the pointer is moved to the coordinates of the press.
- **OnPointerMoved** is an event handler that our app calls when the player swipes a finger across the touch surface. It updates with the new coordinates of the drag path.
- **OnPointerReleased** is an event handler that our app calls when the player removes the pressing finger from the touch surface.

Finally, we use these methods and properties to initialize, access, and update the camera controller state information.

- **Initialize** is an event handler that our app calls to initialize the controls and attach them to the [CoreWindow](#) object that describes your display window.
- **SetPosition** is a method that our app calls to set the (x, y, and z) coordinates of your controls in the scene space. Note that our z-coordinate is 0 throughout this tutorial.
- **get_Position** is a property that our app accesses to get the current position of the camera in the scene space. You use this property as the way of communicating the current camera position to the app.
- **get_FixedLookPoint** is a property that our app accesses to get the current point toward which the controller camera is facing. In this example, it is locked normal to the x-y plane.
- **Update** is a method that reads the controller state and updates the camera position. You continually call this <something> from the app's main loop to refresh the camera controller data and the camera position in the scene space.

Now, you have here all the components you need to implement touch controls. You can detect when and where the touch or mouse pointer events have occurred, and what the action is. You can set the position and orientation of the camera relative to the scene space, and track the changes. Finally, you can communicate the new camera position to the calling app.

Now, let's connect these pieces together.

Create the basic touch events

The Windows Runtime event dispatcher provides 3 events we want our app to handle:

- [PointerPressed](#)
- [PointerMoved](#)
- [PointerReleased](#)

These events are implemented on the [CoreWindow](#) type. We assume that you have a **CoreWindow** object to work with. For more info, see [How to set up your UWP C++ app to display a DirectX view](#).

As these events fire while our app is running, the handlers update the camera controller state info defined in our private fields.

First, let's populate the touch pointer event handlers. In the first event handler, **OnPointerPressed**, we get the x-y coordinates of the pointer from the [CoreWindow](#) that manages our display when the user touches the screen or clicks the mouse.

OnPointerPressed

C++

```
void CameraPanController::OnPointerPressed(
    _In_ CoreWindow^ sender,
    _In_ PointerEventArgs^ args)
{
    // Get the current pointer position.
    uint32 pointerID = args->CurrentPoint->PointerId;
    DirectX::XMFLOAT2 position = DirectX::XMFLOAT2( args->CurrentPoint-
    >Position.X, args->CurrentPoint->Position.Y );

    auto device = args->CurrentPoint->PointerDevice;
    auto deviceType = device->PointerDeviceType;

    if ( !m_panInUse ) // If no pointer is in this control yet.
    {
        m_panFirstDown = position; // Save the location of
        the initial contact.
        m_panPointerPosition = position;
        m_panPointerID = pointerID; // Store the id of the
        pointer using this control.
        m_panInUse = TRUE;
    }
}
```

We use this handler to let the current **CameraPanController** instance know that camera controller should be treated as active by setting **m_panInUse** to TRUE. That way, when the app calls **Update** , it will use the current position data to update the viewport.

Now that we've established the base values for the camera movement when the user touches the screen or click-presses in the display window, we must determine what to do when the user either drags the screen press or moves the mouse with button pressed.

The **OnPointerMoved** event handler fires whenever the pointer moves, at every tick that the player drags it on the screen. We need to keep the app aware of the current location of the pointer, and this is how we do it.

OnPointerMoved

C++

```
void CameraPanController::OnPointerMoved(
    _In_ CoreWindow ^sender,
    _In_ PointerEventArgs ^args)
{
    uint32 pointerID = args->CurrentPoint->PointerId;
    DirectX::XMFLOAT2 position = DirectX::XMFLOAT2( args->CurrentPoint-
```

```

>Position.X, args->CurrentPoint->Position.Y );

    m_panPointerPosition = position;
}

```

Finally, we need to deactivate the camera pan behavior when the player stops touching the screen. We use **OnPointerReleased**, which is called when **PointerReleased** is fired, to set **m_panInUse** to **FALSE** and turn off the camera pan movement, and set the pointer ID to 0.

OnPointerReleased

```

C++

void CameraPanController::OnPointerReleased(
    _In_ CoreWindow ^sender,
    _In_ PointerEventArgs ^args)
{
    uint32 pointerID = args->CurrentPoint->PointerId;
    DirectX::XMFLLOAT2 position = DirectX::XMFLLOAT2( args->CurrentPoint-
>Position.X, args->CurrentPoint->Position.Y );

    m_panInUse = FALSE;
    m_panPointerID = 0;
}

```

Initialize the touch controls and the controller state

Let's hook the events and initialize all the basic state fields of the camera controller.

Initialize

```

C++

void CameraPanController::Initialize( _In_ CoreWindow^ window )
{

    // Start receiving touch/mouse events.
    window->PointerPressed +=
        ref new TypedEventHandler<CoreWindow^, PointerEventArgs^>(this,
&CameraPanController::OnPointerPressed);

    window->PointerMoved +=
        ref new TypedEventHandler<CoreWindow^, PointerEventArgs^>(this,
&CameraPanController::OnPointerMoved);

    window->PointerReleased +=

```



```

    ref new TypedEventHandler<CoreWindow^, PointerEventArgs^>(this,
    &CameraPanController::OnPointerReleased);

    // Initialize the state of the controller.
    m_panInUse = FALSE;
    m_panPointerID = 0;

    // Initialize this as it is reset on every frame.
    m_panCommand = DirectX::XMFLOAT3( 0.0f, 0.0f, 0.0f );

}

```

Initialize takes a reference to the app's **CoreWindow** instance as a parameter and registers the event handlers we developed to the appropriate events on that **CoreWindow**.

Getting and setting the position of the camera controller

Let's define some methods to get and set the position of the camera controller in the scene space.

C++

```

void CameraPanController::SetPosition( _In_ DirectX::XMFLOAT3 pos )
{
    m_position = pos;
}

// Returns the position of the controller object
DirectX::XMFLOAT3 CameraPanController::get_Position()
{
    return m_position;
}

DirectX::XMFLOAT3 CameraPanController::get_FixedLookPoint()
{
    // For this sample, we don't need to use the trig functions because our
    // look point is fixed.
    DirectX::XMFLOAT3 result= m_position;
    result.z += 1.0f;
    return result;
}

```

SetPosition is a public method that we can call from our app if we need to set the camera controller position to a specific point.

get_Position is our most important public property: it's the way our app gets the current position of the camera controller in the scene space so it can update the viewport accordingly.

get_FixedLookPoint is a public property that, in this example, obtains a look point that is normal to the x-y plane. You can change this method to use the trigonometric functions, sin and cos, when calculating the x, y, and z coordinate values if you want to create more oblique angles for the fixed camera.

Updating the camera controller state information

Now, we perform our calculations that convert the pointer coordinate info tracked in **m_panPointerPosition** into new coordinate info respective of our 3D scene space. Our app calls this method every time we refresh the main app loop. In it we compute the new position information we want to pass to the app which is used to update the view matrix before projection into the viewport.

C++

```
void CameraPanController::Update( CoreWindow ^window )
{
    if ( m_panInUse )
    {
        pointerDelta.x = m_panPointerPosition.x - m_panFirstDown.x;
        pointerDelta.y = m_panPointerPosition.y - m_panFirstDown.y;

        if ( pointerDelta.x > 16.0f )           // Leave 32 pixel-wide dead
spot for being still.
            m_panCommand.x += 1.0f;
        else
            if ( pointerDelta.x < -16.0f )
                m_panCommand.x += -1.0f;

        if ( pointerDelta.y > 16.0f )
            m_panCommand.y += 1.0f;
        else
            if ( pointerDelta.y < -16.0f )
                m_panCommand.y += -1.0f;
    }

    DirectX::XMFLOAT3 command = m_panCommand;

    // Our velocity is based on the command.
    DirectX::XMFLOAT3 Velocity;
    Velocity.x = command.x;
    Velocity.y = command.y;
```

```

Velocity.z = 0.0f;

// Integrate
m_position.x = m_position.x + Velocity.x;
m_position.y = m_position.y + Velocity.y;
m_position.z = m_position.z + Velocity.z;

// Clear the movement input accumulator for use during the next frame.
m_panCommand = DirectX::XMFLOAT3( 0.0f, 0.0f, 0.0f );

}

```

Because we don't want touch or mouse jitter to make our camera panning jerky, we set a dead zone around the pointer with a diameter of 32 pixels. We also have a velocity value, which in this case is 1:1 with the pixel traversal of the pointer past the dead zone. You can adjust this behavior to slow down or speed up the rate of movement.

Updating the view matrix with the new camera position

We can now obtain a scene space coordinate that our camera is focused on, and which is updated whenever you tell your app to do so (every 60 seconds in the main app loop, for example). This pseudocode suggests the calling behavior you can implement:

C++

```

myCameraPanController->Update( m_window );

// Update the view matrix based on the camera position.
myCamera->MyMethodToComputeViewMatrix(
    myController->get_Position(),          // The position in the 3D scene
space.
    myController->get_FixedLookPoint(),    // The point in the space
we are looking at.
    DirectX::XMFLOAT3( 0, 1, 0 )          // The axis that is
"up" in our space.
);

```

Congratulations! You've implemented a simple set of camera panning touch controls in your game.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Supporting screen orientation (DirectX and C++)

Article • 10/20/2022

Your Universal Windows Platform (UWP) app can support multiple screen orientations when you handle the [DisplayInformation::OrientationChanged](#) event. Here, we'll discuss best practices for handling screen rotation in your UWP DirectX app, so that the Windows 10 device's graphics hardware are used efficiently and effectively.

Before you start, remember that graphics hardware always outputs pixel data in the same way, regardless of the orientation of the device. Windows 10 devices can determine their current display orientation (with some sort of sensor, or with a software toggle) and allow users to change the display settings. Because of this, Windows 10 itself handles the rotation of the images to ensure they are "upright" based on the orientation of the device. By default, your app receives the notification that something has changed in orientation, for example, a window size. When this happens, Windows 10 immediately rotates the image for final display. For three of the four specific screen orientations (discussed later), Windows 10 uses additional graphic resources and computation to display the final image.

For UWP DirectX apps, the [DisplayInformation](#) object provides basic display orientation data that your app can query. The default orientation is *landscape*, where the pixel width of the display is greater than the height; the alternative orientation is *portrait*, where the display is rotated 90 degrees in either direction and the width becomes less than the height.

Windows 10 defines four specific display orientation modes:

- Landscape—the default display orientation for Windows 10, and is considered the base or identity angle for rotation (0 degrees).
- Portrait—the display has been rotated clockwise 90 degrees (or counter-clockwise 270 degrees).
- Landscape, flipped—the display has been rotated 180 degrees (turned upside-down).
- Portrait, flipped—the display has been rotated clockwise 270 degrees (or counter-clockwise 90 degrees).

When the display rotates from one orientation to another, Windows 10 internally performs a rotation operation to align the drawn image with the new orientation, and the user sees an upright image on the screen.

Also, Windows 10 displays automatic transition animations to create a smooth user experience when shifting from one orientation to another. As the display orientation shifts, the user sees these shifts as a fixed zoom and rotation animation of the displayed screen image. Time is allocated by Windows 10 to the app for layout in the new orientation.

Overall, this is the general process for handling changes in screen orientation:

1. Use a combination of the window bounds values and the display orientation data to keep the swap chain aligned with the native display orientation of the device.
2. Notify Windows 10 of the orientation of the swap chain using [IDXGISwapChain1::SetRotation](#).
3. Change the rendering code to generate images aligned with the user orientation of the device.

Resizing the swap chain and pre-rotating its contents

To perform a basic display resize and pre-rotate its contents in your UWP DirectX app , implement these steps:

1. Handle the [DisplayInformation::OrientationChanged](#) event.
2. Resize the swap chain to the new dimensions of the window.
3. Call [IDXGISwapChain1::SetRotation](#) to set the orientation of the swap chain.
4. Recreate any window size dependent resources, such as your render targets and other pixel data buffers.

Now's let's look at those steps in a bit more detail.

Your first step is to register a handler for the [DisplayInformation::OrientationChanged](#) event. This event is raised in your app every time the screen orientation changes, such as when the display is rotated.

To handle the [DisplayInformation::OrientationChanged](#) event, you connect your handler for [DisplayInformation::OrientationChanged](#) in the required [SetWindow](#) method, which is one of the methods of the [IFrameworkView](#) interface that your view provider must implement.

In this code example, the event handler for [DisplayInformation::OrientationChanged](#) is a method called [OnOrientationChanged](#). When [DisplayInformation::OrientationChanged](#) is raised, it in turn calls a method called [SetCurrentOrientation](#) which then calls [CreateWindowSizeDependentResources](#).

C++

```
void App::SetWindow(CoreWindow^ window)
{
    // ... Other UI event handlers assigned here ...

    currentDisplayInformation->OrientationChanged +=
        ref new TypedEventHandler<DisplayInformation^, Object^>(this,
        &App::OnOrientationChanged);

    // ...
}
}
```

C++

```
void App::OnOrientationChanged(DisplayInformation^ sender, Object^ args)
{
    m_deviceResources->SetCurrentOrientation(sender->CurrentOrientation);
    m_main->CreateWindowSizeDependentResources();
}

// This method is called in the event handler for the OrientationChanged
// event.
void DX::DeviceResources::SetCurrentOrientation(DisplayOrientations
currentOrientation)
{
    if (m_currentOrientation != currentOrientation)
    {
        m_currentOrientation = currentOrientation;
        CreateWindowSizeDependentResources();
    }
}
```

Next, you resize the swap chain for the new screen orientation and prepare it to rotate the contents of the graphic pipeline when the rendering is performed. In this example, **DirectXBase::CreateWindowSizeDependentResources** is a method that handles calling `IDXGISwapChain::ResizeBuffers`, setting a 3D and a 2D rotation matrix, calling `SetRotation`, and recreating your resources.

C++

```
void DX::DeviceResources::CreateWindowSizeDependentResources()
{
    // Clear the previous window size specific context.
    ID3D11RenderTargetView* nullViews[] = {nullptr};
    m_d3dContext->OMSetRenderTargets(ARRAYSIZE(nullViews), nullViews,
    nullptr);
    m_d3dRenderTargetView = nullptr;
    m_d2dContext->SetTarget(nullptr);
}
```

```

m_d2dTargetBitmap = nullptr;
m_d3dDepthStencilView = nullptr;
m_d3dContext->Flush();

// Calculate the necessary render target size in pixels.
m_outputSize.Width = DX::ConvertDipsToPixels(m_logicalSize.Width,
m_dpi);
m_outputSize.Height = DX::ConvertDipsToPixels(m_logicalSize.Height,
m_dpi);

// Prevent zero size DirectX content from being created.
m_outputSize.Width = max(m_outputSize.Width, 1);
m_outputSize.Height = max(m_outputSize.Height, 1);

// The width and height of the swap chain must be based on the window's
// natively-oriented width and height. If the window is not in the
native
// orientation, the dimensions must be reversed.
DXGI_MODE_ROTATION displayRotation = ComputeDisplayRotation();

bool swapDimensions = displayRotation == DXGI_MODE_ROTATION_ROTATE90 ||
displayRotation == DXGI_MODE_ROTATION_ROTATE270;
m_d3dRenderTargetSize.Width = swapDimensions ? m_outputSize.Height :
m_outputSize.Width;
m_d3dRenderTargetSize.Height = swapDimensions ? m_outputSize.Width :
m_outputSize.Height;

if (m_swapChain != nullptr)
{
    // If the swap chain already exists, resize it.
    HRESULT hr = m_swapChain->ResizeBuffers(
        2, // Double-buffered swap chain.
        lround(m_d3dRenderTargetSize.Width),
        lround(m_d3dRenderTargetSize.Height),
        DXGI_FORMAT_B8G8R8A8_UNORM,
        0
    );

    if (hr == DXGI_ERROR_DEVICE_REMOVED || hr ==
DXGI_ERROR_DEVICE_RESET)
    {
        // If the device was removed for any reason, a new device and
swap chain will need to be created.
        HandleDeviceLost();

        // Everything is set up now. Do not continue execution of this
method. HandleDeviceLost will reenter this method
        // and correctly set up the new device.
        return;
    }
    else
    {
        DX::ThrowIfFailed(hr);
    }
}
}

```



```

else
{
    // Otherwise, create a new one using the same adapter as the
    existing Direct3D device.
    DXGI_SWAP_CHAIN_DESC1 swapChainDesc = {0};

    swapChainDesc.Width = lround(m_d3dRenderTargetSize.Width); // Match
    the size of the window.
    swapChainDesc.Height = lround(m_d3dRenderTargetSize.Height);
    swapChainDesc.Format = DXGI_FORMAT_B8G8R8A8_UNORM; // This is the
    most common swap chain format.
    swapChainDesc.Stereo = false;
    swapChainDesc.SampleDesc.Count = 1; // Don't use multi-sampling.
    swapChainDesc.SampleDesc.Quality = 0;
    swapChainDesc.BufferUsage = DXGI_USAGE_RENDER_TARGET_OUTPUT;
    swapChainDesc.BufferCount = 2; // Use double-buffering to minimize
    latency.
    swapChainDesc.SwapEffect = DXGI_SWAP_EFFECT_FLIP_SEQUENTIAL; // All
    UWP apps must use this SwapEffect.
    swapChainDesc.Flags = 0;
    swapChainDesc.Scaling = DXGI_SCALING_NONE;
    swapChainDesc.AlphaMode = DXGI_ALPHA_MODE_IGNORE;

    // This sequence obtains the DXGI factory that was used to create
    the Direct3D device above.
    ComPtr<IDXGIDevice3> dxgiDevice;
    DX::ThrowIfFailed(
        m_d3dDevice.As(&dxgiDevice)
    );

    ComPtr<IDXGIAdapter> dxgiAdapter;
    DX::ThrowIfFailed(
        dxgiDevice->GetAdapter(&dxgiAdapter)
    );

    ComPtr<IDXGIFactory2> dxgiFactory;
    DX::ThrowIfFailed(
        dxgiAdapter->GetParent(IID_PPV_ARGS(&dxgiFactory))
    );

    DX::ThrowIfFailed(
        dxgiFactory->CreateSwapChainForCoreWindow(
            m_d3dDevice.Get(),
            reinterpret_cast<IUnknown*>(m_window.Get()),
            &swapChainDesc,
            nullptr,
            &m_swapChain
        )
    );

    // Ensure that DXGI does not queue more than one frame at a time.
    This both reduces latency and
    // ensures that the application will only render after each VSync,
    minimizing power consumption.
    DX::ThrowIfFailed(

```

```

        dxgiDevice->SetMaximumFrameLatency(1)
    );
}

// Set the proper orientation for the swap chain, and generate 2D and
// 3D matrix transformations for rendering to the rotated swap chain.
// Note the rotation angle for the 2D and 3D transforms are different.
// This is due to the difference in coordinate spaces. Additionally,
// the 3D matrix is specified explicitly to avoid rounding errors.

switch (displayRotation)
{
case DXGI_MODE_ROTATION_IDENTITY:
    m_orientationTransform2D = Matrix3x2F::Identity();
    m_orientationTransform3D = ScreenRotation::Rotation0;
    break;

case DXGI_MODE_ROTATION_ROTATE90:
    m_orientationTransform2D =
        Matrix3x2F::Rotation(90.0f) *
        Matrix3x2F::Translation(m_logicalSize.Height, 0.0f);
    m_orientationTransform3D = ScreenRotation::Rotation270;
    break;

case DXGI_MODE_ROTATION_ROTATE180:
    m_orientationTransform2D =
        Matrix3x2F::Rotation(180.0f) *
        Matrix3x2F::Translation(m_logicalSize.Width,
m_logicalSize.Height);
    m_orientationTransform3D = ScreenRotation::Rotation180;
    break;

case DXGI_MODE_ROTATION_ROTATE270:
    m_orientationTransform2D =
        Matrix3x2F::Rotation(270.0f) *
        Matrix3x2F::Translation(0.0f, m_logicalSize.Width);
    m_orientationTransform3D = ScreenRotation::Rotation90;
    break;

default:
    throw ref new FailureException();
}

//SDM: only instance of SetRotation
DX::ThrowIfFailed(
    m_swapChain->SetRotation(displayRotation)
);

// Create a render target view of the swap chain back buffer.
ComPtr<ID3D11Texture2D> backBuffer;
DX::ThrowIfFailed(
    m_swapChain->GetBuffer(0, IID_PPV_ARGS(&backBuffer))
);

```

```

DX::ThrowIfFailed(
    m_d3dDevice->CreateRenderTargetView(
        backBuffer.Get(),
        nullptr,
        &m_d3dRenderTargetView
    )
);

// Create a depth stencil view for use with 3D rendering if needed.
CD3D11_TEXTURE2D_DESC depthStencilDesc(
    DXGI_FORMAT_D24_UNORM_S8_UINT,
    lround(m_d3dRenderTargetSize.Width),
    lround(m_d3dRenderTargetSize.Height),
    1, // This depth stencil view has only one texture.
    1, // Use a single mipmap level.
    D3D11_BIND_DEPTH_STENCIL
);

ComPtr<ID3D11Texture2D> depthStencil;
DX::ThrowIfFailed(
    m_d3dDevice->CreateTexture2D(
        &depthStencilDesc,
        nullptr,
        &depthStencil
    )
);

CD3D11_DEPTH_STENCIL_VIEW_DESC
depthStencilViewDesc(D3D11_DSV_DIMENSION_TEXTURE2D);
DX::ThrowIfFailed(
    m_d3dDevice->CreateDepthStencilView(
        depthStencil.Get(),
        &depthStencilViewDesc,
        &m_d3dDepthStencilView
    )
);

// Set the 3D rendering viewport to target the entire window.
m_screenViewport = CD3D11_VIEWPORT(
    0.0f,
    0.0f,
    m_d3dRenderTargetSize.Width,
    m_d3dRenderTargetSize.Height
);

m_d3dContext->RSSetViewports(1, &m_screenViewport);

// Create a Direct2D target bitmap associated with the
// swap chain back buffer and set it as the current target.
D2D1_BITMAP_PROPERTIES1 bitmapProperties =
    D2D1::BitmapProperties1(
        D2D1_BITMAP_OPTIONS_TARGET | D2D1_BITMAP_OPTIONS_CANNOT_DRAW,
        D2D1::PixelFormat(DXGI_FORMAT_B8G8R8A8_UNORM,
        D2D1_ALPHA_MODE_PREMULTIPLIED),
        m_dpi,

```

```

        m_dpi
    );

    ComPtr<IDXGISurface2> dxgiBackBuffer;
    DX::ThrowIfFailed(
        m_swapChain->GetBuffer(0, IID_PPV_ARGS(&dxgiBackBuffer))
    );

    DX::ThrowIfFailed(
        m_d2dContext->CreateBitmapFromDxgiSurface(
            dxgiBackBuffer.Get(),
            &bitmapProperties,
            &m_d2dTargetBitmap
        )
    );

    m_d2dContext->SetTarget(m_d2dTargetBitmap.Get());

    // Grayscale text anti-aliasing is recommended for all UWP apps.
    m_d2dContext->SetTextAntialiasMode(D2D1_TEXT_ANTIALIAS_MODE_GRAYSCALE);
}

```

After saving the current height and width values of the window for the next time this method is called, convert the device independent pixel (DIP) values for the display bounds to pixels. In the sample, you call **ConvertDipsToPixels**, which is a simple function that runs this code:

```

floor((dips * dpi / 96.0f) + 0.5f);

```

You add the 0.5f to ensure rounding to the nearest integer value.

As an aside, **CoreWindow** coordinates are always defined in DIPs. For Windows 10 and earlier versions of Windows, a DIP is defined as 1/96th of an inch, and aligned to the OS's definition of *up*. When the display orientation rotates to portrait mode, the app flips the width and height of the **CoreWindow**, and the render target size (bounds) must change accordingly. Because Direct3D's coordinates are always in physical pixels, you must convert from **CoreWindow**'s DIP values to integer pixel values before you pass these values to Direct3D to set up the swap chain.

Process-wise, you're doing a bit more work than you would if you simply resized the swap chain: you're actually rotating the Direct2D and Direct3D components of your image before you composite them for presentation, and you're telling the swap chain that you've rendered the results in a new orientation. Here's a little more detail on this process, as shown in the code example for

DX::DeviceResources::CreateWindowSizeDependentResources:

- Determine the new orientation of the display. If the display has flipped from landscape to portrait, or vice versa, swap the height and width values—changed from DIP values to pixels, of course—for the display bounds.
- Then, check to see if the swap chain has been created. If it hasn't been created, create it by calling [IDXGIFactory2::CreateSwapChainForCoreWindow](#). Otherwise, resize the existing swap chain's buffers to the new display dimensions by calling [IDXGISwapchain:ResizeBuffers](#). Although you don't need to resize the swap chain for the rotation event—you're outputting the content already rotated by your rendering pipeline, after all—there are other size change events, such as snap and fill events, that require resizing.
- After that, set the appropriate 2-D or 3-D matrix transformation to apply to the pixels or the vertices (respectively) in the graphics pipeline when rendering them to the swap chain. We have 4 possible rotation matrices:
 - landscape (DXGI_MODE_ROTATION_IDENTITY)
 - portrait (DXGI_MODE_ROTATION_ROTATE270)
 - landscape, flipped (DXGI_MODE_ROTATION_ROTATE180)
 - portrait, flipped (DXGI_MODE_ROTATION_ROTATE90)

The correct matrix is selected based on the data provided by Windows 10 (such as the results of [DisplayInformation::OrientationChanged](#)) for determining display orientation, and it will be multiplied by the coordinates of each pixel (Direct2D) or vertex (Direct3D) in the scene, effectively rotating them to align to the orientation of the screen. (Note that in Direct2D, the screen origin is defined as the upper-left corner, while in Direct3D the origin is defined as the logical center of the window.)

Note For more info about the 2-D transformations used for rotation and how to define them, see [Defining matrices for screen rotation \(2-D\)](#). For more info about the 3-D transformations used for rotation, see [Defining matrices for screen rotation \(3-D\)](#).

Now, here's the important bit: call [IDXGISwapChain1::SetRotation](#) and provide it with your updated rotation matrix, like this:

```
m_swapChain->SetRotation(rotation);
```

You also store the selected rotation matrix where your render method can get it when it computes the new projection. You'll use this matrix when you render your final 3-D projection or composite your final 2-D layout. (It doesn't automatically apply it for you.)

After that, create a new render target for the rotated 3-D view, as well as a new depth stencil buffer for the view. Set the 3-D rendering viewport for the rotated scene by calling [ID3D11DeviceContext::RSSetViewports](#).

Lastly, if you have 2-D images to rotate or lay out, create a 2-D render target as a writable bitmap for the resized swap chain using [ID2D1DeviceContext::CreateBitmapFromDxgiSurface](#) and composite your new layout for the updated orientation. Set any properties you need to on the render target, such as the anti-aliasing mode (as seen in the code example).

Now, present the swap chain.

Reduce the rotation delay by using CoreWindowResizeManager

By default, Windows 10 provides a short but noticeable window of time for any app, regardless of app model or language, to complete the rotation of the image. However, chances are that when your app performs the rotation calculation using one of the techniques described here, it will be done well before this window of time has closed. You'd like to get that time back and complete the rotation animation, right? That's where [CoreWindowResizeManager](#) comes in.

Here's how to use [CoreWindowResizeManager](#): when a [DisplayInformation::OrientationChanged](#) event is raised, call [CoreWindowResizeManager::GetForCurrentView](#) within the handler for the event to obtain an instance of [CoreWindowResizeManager](#) and, when the layout for the new orientation is complete and presented, call the [NotifyLayoutCompleted](#) to let Windows know that it can complete the rotation animation and display the app screen.

Here's what the code in your event handler for [DisplayInformation::OrientationChanged](#) might look like:

C++

```
CoreWindowResizeManager^ resizeMode =  
Windows::UI::Core::CoreWindowResizeManager::GetForCurrentView();  
  
// ... build the layout for the new display orientation ...  
  
resizeManager->NotifyLayoutCompleted();
```

When a user rotates the orientation of the display, Windows 10 shows an animation independent of your app as feedback to the user. There are three parts to that

animation that happen in the following order:

- Windows 10 shrinks the original image.
- Windows 10 holds the image for the time it takes to rebuild the new layout. This is the window of time that you'd like to reduce, because your app probably doesn't need all of it.
- When the layout window expires, or when a notification of layout completion is received, Windows rotates the image and then cross-fade zooms to new orientation.

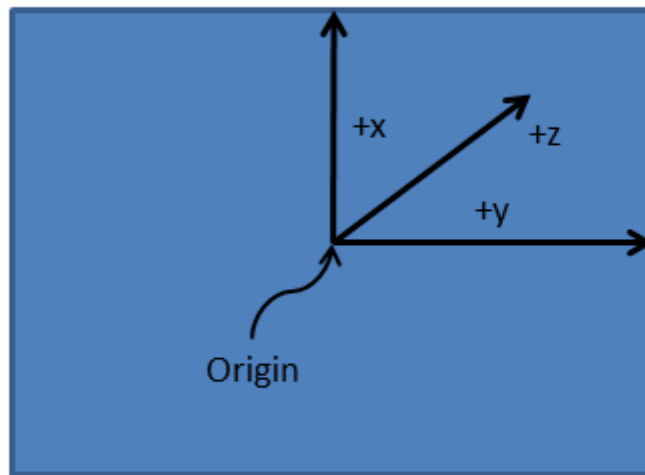
As suggested in the third bullet, when an app calls [NotifyLayoutCompleted](#), Windows 10 stops the timeout window, completes the rotation animation and returns control to your app, which is now drawing in the new display orientation. The overall effect is that your app now feels a little bit more fluid and responsive, and works a little more efficiently!

Appendix A: Applying matrices for screen rotation (2-D)

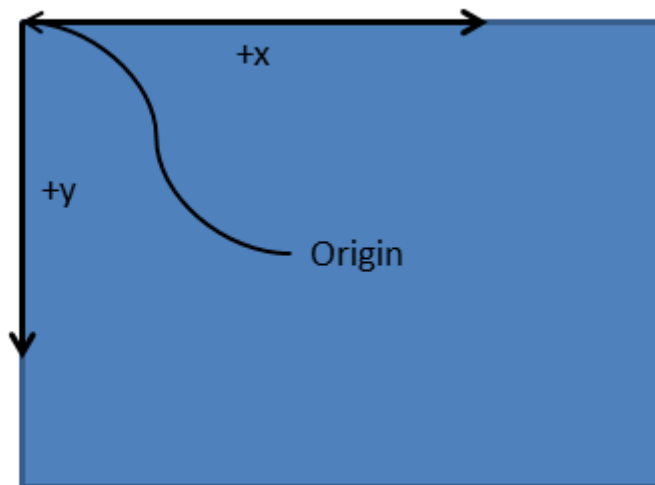
In the example code in [Resizing the swap chain and pre-rotating its contents](#) (and in the [DXGI swap chain rotation sample](#) [↗](#)), you might have noticed that we had separate rotation matrices for Direct2D output and Direct3D output. Let's look at the 2-D matrices, first.

There are two reasons that we can't apply the same rotation matrices to Direct2D and Direct3D content:

- One, they use different Cartesian coordinate models. Direct2D uses the right-handed rule, where the y-coordinate increases in positive value moving upward from the origin. However, Direct3D uses the left-handed rule, where the y-coordinate increases in positive value rightward from the origin. The result is the origin for the screen coordinates is located in the upper-left for Direct2D, while the origin for the screen (the projection plane) is in the lower-left for Direct3D. (See [3-D coordinate systems](#) for more info.)



Direct3D



Direct2D

- Two, the 3-D rotation matrices must be specified explicitly to avoid rounding errors.

The swap chain assumes that the origin is located in the lower-left, so you must perform a rotation to align the right-handed Direct2D coordinate system with the left-handed one used by the swap chain. Specifically, you reposition the image under the new left-handed orientation by multiplying the rotation matrix with a translation matrix for the rotated coordinate system origin, and transform the image from the [CoreWindow's](#) coordinate space to the swap chain's coordinate space. Your app also must consistently apply this transform when the Direct2D render target is connected with the swap chain. However, if your app is drawing to intermediate surfaces that are not associated directly with the swap chain, don't apply this coordinate space transformation.

Your code to select the correct matrix from the four possible rotations might look like this (be aware of the translation to the new coordinate system origin):

C++

```
// Set the proper orientation for the swap chain, and generate 2D and
// 3D matrix transformations for rendering to the rotated swap chain.
// Note the rotation angle for the 2D and 3D transforms are different.
// This is due to the difference in coordinate spaces. Additionally,
// the 3D matrix is specified explicitly to avoid rounding errors.

switch (displayRotation)
{
case DXGI_MODE_ROTATION_IDENTITY:
    m_orientationTransform2D = Matrix3x2F::Identity();
    m_orientationTransform3D = ScreenRotation::Rotation0;
    break;

case DXGI_MODE_ROTATION_ROTATE90:
    m_orientationTransform2D =
        Matrix3x2F::Rotation(90.0f) *
        Matrix3x2F::Translation(m_logicalSize.Height, 0.0f);
    m_orientationTransform3D = ScreenRotation::Rotation270;
    break;

case DXGI_MODE_ROTATION_ROTATE180:
    m_orientationTransform2D =
        Matrix3x2F::Rotation(180.0f) *
        Matrix3x2F::Translation(m_logicalSize.Width, m_logicalSize.Height);
    m_orientationTransform3D = ScreenRotation::Rotation180;
    break;

case DXGI_MODE_ROTATION_ROTATE270:
    m_orientationTransform2D =
        Matrix3x2F::Rotation(270.0f) *
        Matrix3x2F::Translation(0.0f, m_logicalSize.Width);
    m_orientationTransform3D = ScreenRotation::Rotation90;
    break;

default:
    throw ref new FailureException();
}
```

After you have the correct rotation matrix and origin for the 2-D image, set it with a call to [ID2D1DeviceContext::SetTransform](#) between your calls to [ID2D1DeviceContext::BeginDraw](#) and [ID2D1DeviceContext::EndDraw](#).

Warning Direct2D doesn't have a transformation stack. If your app is also using the [ID2D1DeviceContext::SetTransform](#) as a part of its drawing code, this matrix needs to

be post-multiplied to any other transform you have applied.

C++

```
ID2D1DeviceContext* context = m_deviceResources->GetD2DDeviceContext();
Windows::Foundation::Size logicalSize = m_deviceResources-
>GetLogicalSize();

context->SaveDrawingState(m_stateBlock.Get());
context->BeginDraw();

// Position on the bottom right corner.
D2D1::Matrix3x2F screenTranslation = D2D1::Matrix3x2F::Translation(
    logicalSize.Width - m_textMetrics.layoutWidth,
    logicalSize.Height - m_textMetrics.height
);

context->SetTransform(screenTranslation * m_deviceResources-
>GetOrientationTransform2D());

DX::ThrowIfFailed(
    m_textFormat->SetTextAlignment(DWRITE_TEXT_ALIGNMENT_TRAILING)
);

context->DrawTextLayout(
    D2D1::Point2F(0.f, 0.f),
    m_textLayout.Get(),
    m_whiteBrush.Get()
);

// Ignore D2DERR_RECREATE_TARGET here. This error indicates that the
device
// is lost. It will be handled during the next call to Present.
HRESULT hr = context->EndDraw();
```

The next time you present the swap chain, your 2-D image will be rotated to match the new display orientation.

Appendix B: Applying matrices for screen rotation (3-D)

In the example code in [Resizing the swap chain and pre-rotating its contents](#) (and in the [DXGI swap chain rotation sample](#) [↗](#)), we defined a specific transformation matrix for each possible screen orientation. Now, let's look at the matrices for rotating 3-D scenes. As before, you create a set of matrices for each of the 4 possible orientations. To prevent

rounding errors and thus minor visual artifacts, declare the matrices explicitly in your code.

You set up these 3-D rotation matrices as follows. The matrices shown in the following code example are standard rotation matrices for 0, 90, 180, and 270 degree rotations of the vertices that define points in the camera's 3-D scene space. Each vertex's [x, y, z] coordinate value in the scene is multiplied by this rotation matrix when the 2-D projection of the scene is computed.

C++

```
// 0-degree Z-rotation
static const XMFLOAT4X4 Rotation0(
    1.0f, 0.0f, 0.0f, 0.0f,
    0.0f, 1.0f, 0.0f, 0.0f,
    0.0f, 0.0f, 1.0f, 0.0f,
    0.0f, 0.0f, 0.0f, 1.0f
);

// 90-degree Z-rotation
static const XMFLOAT4X4 Rotation90(
    0.0f, 1.0f, 0.0f, 0.0f,
    -1.0f, 0.0f, 0.0f, 0.0f,
    0.0f, 0.0f, 1.0f, 0.0f,
    0.0f, 0.0f, 0.0f, 1.0f
);

// 180-degree Z-rotation
static const XMFLOAT4X4 Rotation180(
    -1.0f, 0.0f, 0.0f, 0.0f,
    0.0f, -1.0f, 0.0f, 0.0f,
    0.0f, 0.0f, 1.0f, 0.0f,
    0.0f, 0.0f, 0.0f, 1.0f
);

// 270-degree Z-rotation
static const XMFLOAT4X4 Rotation270(
    0.0f, -1.0f, 0.0f, 0.0f,
    1.0f, 0.0f, 0.0f, 0.0f,
    0.0f, 0.0f, 1.0f, 0.0f,
    0.0f, 0.0f, 0.0f, 1.0f
);
}
```

You set the rotation type on the swap chain with a call to [IDXGISwapChain1::SetRotation](#), like this:

```
m_swapChain->SetRotation(rotation);
```

Now, in your render method, implement some code similar to this:

syntax

```
struct ConstantBuffer // This struct is provided for illustration.
{
    // Other constant buffer matrices and data are defined here.

    float4x4 projection; // Current matrix for projection
} ;
ConstantBuffer m_constantBufferData;          // Constant buffer resource
data

// ...

// Rotate the projection matrix as it will be used to render to the rotated
swap chain.
m_constantBufferData.projection = mul(m_constantBufferData.projection,
m_rotationTransform3D);
```

Now, when you call your render method, it multiplies the current rotation matrix (as specified by the class variable **m_orientationTransform3D**) with the current projection matrix, and assigns the results of that operation as the new projection matrix for your renderer. Present the swap chain to see the scene in the updated display orientation.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Optimization and advanced topics for DirectX games

Article • 10/20/2022

This section provides information about optimizing your DirectX game performance and other advanced topics.

Asynchronous programming for games topic discusses the various points to consider when you want to use asynchronous programming to parallelize some of the components and use multithreading to maximise the use of a powerful GPU.

Handle device removed scenarios in Direct3D 11 topic uses a walkthrough to explain how games developed using Direct3D 11 can detect and respond to situations where the graphics adapter is reset, removed, or changed.

Multisampling in UWP apps topic provides an overview of how to use multi-sample antialiasing, a graphics technique to reduce the appearance of aliased edges in UWP games built with Direct3D.

Optimize input and rendering loop topic provides guidance on how to choose the right input event processing option to manage input latency and optimize the rendering loop.

Reduce latency with DXGI 1.3 swap chains topic explains how to reduce effective frame latency by waiting for the swap chain to signal the appropriate time to begin rendering a new frame.

Swap chain scaling and overlays topic explains how to improve rendering times by using scaled swap chains to render real-time game content at a lower resolution than the display is natively capable of. It also explains how to create overlay swap chains for devices with the hardware overlay capability; this technique can be used to alleviate the issue of a scaled down UI resulting from the use of swap chain scaling.

Topic	Description
Asynchronous programming for games	Understand asynchronous programming and threading with DirectX.
Handle device removed scenarios in Direct3D 11	Recreate the Direct3D and DXGI device interface chain when the graphics adapter is removed or reinitialized.

Topic	Description
Multisampling in UWP apps	Use multisampling in UWP games built using Direct3D.
Optimize input and rendering loop	Reduce input latency and optimize the rendering loop.
Reduce latency with DXGI 1.3 swap chains	Use DXGI 1.3 to reduce the effective frame latency.
Swap chain scaling and overlays	Create scaled swap chains for faster rendering on mobile devices, and use overlay swap chains to increase the visual quality.

Asynchronous programming (DirectX and C++)

Article • 10/27/2022

This topic covers various points to consider when you are using asynchronous programming and threading with DirectX.

Async programming and DirectX

If you're just learning about DirectX, or even if you're experienced with it, consider putting all your graphics processing pipeline on one thread. In any given scene in a game, there are common resources such as bitmaps, shaders, and other assets that require exclusive access. These same resources require that you synchronize any access to these resources across the parallel threads. Rendering is a difficult process to parallelize across multiple threads.

However, if your game is sufficiently complex, or if you are looking to get improved performance, you can use asynchronous programming to parallelize some of the components that are not specific to your rendering pipeline. Modern hardware features multiple core and hyperthreaded CPUs, and your app should take advantage of this! You can ensure this by using asynchronous programming for some of the components of your game that don't need direct access to the Direct3D device context, such as:

- file I/O
- physics
- AI
- networking
- audio
- controls
- XAML-based UI components

Your app can handle these components on multiple concurrent threads. File I/O, especially asset loading, benefits greatly from asynchronous loading, because your game or app can be in an interactive state while several (or several hundred) megabytes of assets are being loaded or streamed. The easiest way to create and manage these threads is by using the [Parallel Patterns Library](#) and the `task` pattern, as contained in the `concurrency` namespace defined in `PPLTasks.h`. Using the [Parallel Patterns Library](#) takes direct advantage of multiple core and hyperthreaded CPUs, and can improve everything from perceived load times to the hitches and lags that come with intensive CPU calculations or network processing.

Note In a Universal Windows Platform (UWP) app, the user interface runs entirely in a single-threaded apartment (STA). If you are creating a UI for your DirectX game using [XAML interop](#), you can only access the controls by using the STA.

Multithreading with Direct3D devices

Multithreading for device contexts is only available on graphics devices that support a Direct3D feature level of 11_0 or later. However, you might want to maximize the use of the powerful GPU in many platforms, such as dedicated gaming platforms. In the simplest case, you might want to separate the rendering of a heads-up display (HUD) overlay from the 3D scene rendering and projection and have both components use separate parallel pipelines. Both threads must use the same [ID3D11DeviceContext](#) to create and manage the resource objects (the textures, meshes, shaders, and other assets), though, which is single-threaded, and which requires that you implement some sort of synchronization mechanism (such as critical sections) to access it safely. And, while you can create separate command lists for the device context on different threads (for deferred rendering), you cannot play those command lists back simultaneously on the same [ID3D11DeviceContext](#) instance.

Now, your app can also use [ID3D11Device](#), which is safe for multithreading, to create resource objects. So, why not always use [ID3D11Device](#) instead of [ID3D11DeviceContext](#)? Well, currently, driver support for multithreading might not be available for some graphics interfaces. You can query the device and find out if it does support multithreading, but if you are looking to reach the broadest audience, you might stick with single-threaded [ID3D11DeviceContext](#) for resource object management. That said, when the graphics device driver doesn't support multithreading or command lists, Direct3D 11 attempts to handle synchronized access to the device context internally; and if command lists are not supported, it provides a software implementation. As a result, you can write multithreaded code that will run on platforms with graphics interfaces that lack driver support for multithreaded device context access.

If your app supports separate threads for processing command lists and for displaying frames, you probably want to keep the GPU active, processing the command lists while displaying frames in a timely fashion without perceptible stutter or lag. In this case, you could use a separate [ID3D11DeviceContext](#) for each thread, and to share resources (like textures) by creating them with the `D3D11_RESOURCE_MISC_SHARED` flag. In this scenario, [ID3D11DeviceContext::Flush](#) must be called on the processing thread to complete the execution of the command list prior to displaying the results of processing the resource object in the display thread.

Deferred rendering

Deferred rendering records graphics commands in a command list so that they can be played back at some other time, and is designed to support rendering on one thread while recording commands for rendering on additional threads. After these commands are completed, they can be executed on the thread that generates the final display object (frame buffer, texture, or other graphics output).

Create a deferred context using [ID3D11Device::CreateDeferredContext](#) (instead of [D3D11CreateDevice](#) or [D3D11CreateDeviceAndSwapChain](#), which create an immediate context). For more info, see [Immediate and Deferred Rendering](#).

Related topics

- [Introduction to Multithreading in Direct3D 11](#)

Handle device removed scenarios in Direct3D 11

Article • 10/20/2022

This topic explains how to recreate the Direct3D and DXGI device interface chain when the graphics adapter is removed or reinitialized.

In DirectX 9, applications could encounter a "[device lost](#)" condition where the D3D device enters a non-operational state. For example, when a full-screen Direct3D 9 application loses focus, the Direct3D device becomes "lost;" any attempts to draw with a lost device will silently fail. Direct3D 11 uses virtual graphics device interfaces, enabling multiple programs to share the same physical graphics device and eliminating conditions where apps lose control of the Direct3D device. However, it is still possible for graphics adapter availability to change. For example:

- The graphics driver is upgraded.
- The system changes from a power-saving graphics adapter to a performance graphics adapter.
- The graphics device stops responding and is reset.
- A graphics adapter is physically attached or removed.

When such circumstances arise, DXGI returns an error code indicating that the Direct3D device must be reinitialized and device resources must be recreated. This walkthrough explains how Direct3D 11 apps and games can detect and respond to any circumstance where the graphics adapter is reset, removed, or changed. Code examples are provided from the DirectX 11 App (Universal Windows) template provided with Microsoft Visual Studio 2015.

Instructions

Step 1:

Include a check for the device removed error in the rendering loop. Present the frame by calling [IDXGISwapChain::Present](#) (or [Present1](#), and so on). Then, check whether it returned [DXGI_ERROR_DEVICE_REMOVED](#) or [DXGI_ERROR_DEVICE_RESET](#).

First, the template stores the HRESULT returned by the DXGI swap chain:

```
C++
```

```
HRESULT hr = m_swapChain->Present(1, 0);
```

After taking care of all other work for presenting the frame, the template checks for the device removed error. If necessary, it calls a method to handle the device removed condition:

C++

```
// If the device was removed either by a disconnection or a driver upgrade,
// we
// must recreate all device resources.
if (hr == DXGI_ERROR_DEVICE_REMOVED || hr == DXGI_ERROR_DEVICE_RESET)
{
    HandleDeviceLost();
}
else
{
    DX::ThrowIfFailed(hr);
}
```

Step 2:

Also, include a check for the device removed error when responding to window size changes. This is a good place to check for [DXGI_ERROR_DEVICE_REMOVED](#) or [DXGI_ERROR_DEVICE_RESET](#) for several reasons:

- Resizing the swap chain requires a call to the underlying DXGI adapter, which can return the device removed error.
- The app might have moved to a monitor that's attached to a different graphics device.
- When a graphics device is removed or reset, the desktop resolution often changes, resulting in a window size change.

The template checks the HRESULT returned by [ResizeBuffers](#):

C++

```
// If the swap chain already exists, resize it.
HRESULT hr = m_swapChain->ResizeBuffers(
    2, // Double-buffered swap chain.
    static_cast<UINT>(m_d3dRenderTargetSize.Width),
    static_cast<UINT>(m_d3dRenderTargetSize.Height),
    DXGI_FORMAT_B8G8R8A8_UNORM,
    0
);
```

```

if (hr == DXGI_ERROR_DEVICE_REMOVED || hr == DXGI_ERROR_DEVICE_RESET)
{
    // If the device was removed for any reason, a new device and swap chain
    will need to be created.
    HandleDeviceLost();

    // Everything is set up now. Do not continue execution of this method.
    HandleDeviceLost will reenter this method
    // and correctly set up the new device.
    return;
}
else
{
    DX::ThrowIfFailed(hr);
}

```

Step 3:

Any time your app receives the [DXGI_ERROR_DEVICE_REMOVED](#) error, it must reinitialize the Direct3D device and recreate any device-dependent resources. Release any references to graphics device resources created with the previous Direct3D device; those resources are now invalid, and all references to the swap chain must be released before a new one can be created.

The `HandleDeviceLost` method releases the swap chain and notifies app components to release device resources:

C++

```

m_swapChain = nullptr;

if (m_deviceNotify != nullptr)
{
    // Notify the renderers that device resources need to be released.
    // This ensures all references to the existing swap chain are released
    so that a new one can be created.
    m_deviceNotify->OnDeviceLost();
}

```

Then, it creates a new swap chain and reinitializes the device-dependent resources controlled by the device management class:

C++

```

// Create the new device and swap chain.
CreateDeviceResources();
m_d2dContext->SetDpi(m_dpi, m_dpi);
CreateWindowSizeDependentResources();

```

After the device and swap chain have been re-established, it notifies app components to reinitialize device-dependent resources:

C++

```
// Create the new device and swap chain.
CreateDeviceResources();
m_d2dContext->SetDpi(m_dpi, m_dpi);
CreateWindowSizeDependentResources();

if (m_deviceNotify != nullptr)
{
    // Notify the renderers that resources can now be created again.
    m_deviceNotify->OnDeviceRestored();
}
```

When the `HandleDeviceLost` method exits, control returns to the rendering loop, which continues on to draw the next frame.

Remarks

Investigating the cause of device removed errors

Repeat issues with DXGI device removed errors can indicate that your graphics code is creating invalid conditions during a drawing routine. It can also indicate a hardware failure or a bug in the graphics driver. To investigate the cause of device removed errors, call [ID3D11Device::GetDeviceRemovedReason](#) before releasing the Direct3D device. This method returns one of six possible DXGI error codes indicating the reason for the device removed error:

- **DXGI_ERROR_DEVICE_HUNG**: The graphics driver stopped responding because of an invalid combination of graphics commands sent by the app. If you get this error repeatedly, it is a likely indication that your app caused the device to hang and needs to be debugged.
- **DXGI_ERROR_DEVICE_REMOVED**: The graphics device has been physically removed, turned off, or a driver upgrade has occurred. This happens occasionally and is normal; your app or game should recreate device resources as described in this topic.
- **DXGI_ERROR_DEVICE_RESET**: The graphics device failed because of a badly formed command. If you get this error repeatedly, it may mean that your code is sending invalid drawing commands.

- **DXGI_ERROR_DRIVER_INTERNAL_ERROR**: The graphics driver encountered an error and reset the device.
- **DXGI_ERROR_INVALID_CALL**: The application provided invalid parameter data. If you get this error even once, it means that your code caused the device removed condition and must be debugged.
- **S_OK**: Returned when a graphics device was enabled, disabled, or reset without invalidating the current graphics device. For example, this error code can be returned if an app is using [Windows Advanced Rasterization Platform \(WARP\)](#) and a hardware adapter becomes available.

The following code will retrieve the **DXGI_ERROR_DEVICE_REMOVED** error code and print it to the debug console. Insert this code at the beginning of the `HandleDeviceLost` method:

C++

```
HRESULT reason = m_d3dDevice->GetDeviceRemovedReason();

#ifdef _DEBUG
wchar_t outString[100];
size_t size = 100;
swprintf_s(outString, size, L"Device removed! DXGI_ERROR code: 0x%X\n",
reason);
OutputDebugStringW(outString);
#endif
```

For more details, see [GetDeviceRemovedReason](#) and [DXGI_ERROR](#).

Testing Device Removed Handling

Visual Studio's Developer Command Prompt supports a command line tool 'dxcap' for Direct3D event capture and playback related to the Visual Studio Graphics Diagnostics. You can use the command line option "-forcetdr" while your app is running which will force a GPU Timeout Detection and Recovery event, thereby triggering **DXGI_ERROR_DEVICE_REMOVED** and allowing you to test your error handling code.

Note DXCap and its support DLLs are installed into system32/syswow64 as part of the Graphics Tools for Windows 10 which are no longer distributed via the Windows SDK. Instead they are provided via the Graphics Tools Feature on Demand that is an optional OS component and must be installed in order to enable and use the Graphics Tools on Windows 10. More information on how to Install the Graphics Tools for Windows 10 can be found here:

<https://msdn.microsoft.com/library/mt125501.aspx#InstallGraphicsTools> 

Multisampling in Universal Windows Platform (UWP) apps

Article • 10/20/2022

Learn how to use multisampling in Universal Windows Platform (UWP) apps built with Direct3D. Multisampling, also known as multi-sample antialiasing, is a graphics technique used to reduce the appearance of aliased edges. It works by drawing more pixels than are actually in the final render target, then averaging values to maintain the appearance of a "partial" edge in certain pixels. For a detailed description of how multisampling actually works in Direct3D, see [Multisample Anti-Aliasing Rasterization Rules](#).

Multisampling and the flip model swap chain

UWP apps that use DirectX must use flip model swap chains. Flip model swap chains don't support multisampling directly, but multisampling can still be applied in a different way by rendering the scene to a multisampled render target view, and then resolving the multisampled render target to the back buffer before presenting. This article explains the steps required to add multisampling to your UWP app.

How to use multisampling

Direct3D feature levels guarantee support for specific, minimum sample count capabilities, and guarantee certain buffer formats will be available that support multisampling. Graphics devices often support a wider range of formats and sample counts than the minimum required. Multisampling support can be determined at run-time by checking feature support for multisampling with specific DXGI formats, and then checking the sample counts you can use with each supported format.

1. Call [ID3D11Device::CheckFeatureSupport](#) to find out which DXGI formats can be used with multisampling. Supply the render target formats your game can use. Both the render target and resolve target must use the same format, so check for both [D3D11_FORMAT_SUPPORT_MULTISAMPLE_RENDER_TARGET](#) and [D3D11_FORMAT_SUPPORT_MULTISAMPLE_RESOLVE](#).

****Feature level 9:** Although feature level 9 devices [guarantee support for multisampled render target formats](#), support is not guaranteed for multisample resolve targets. So this check is necessary before trying to use the multisampling technique described in this topic.

The following code checks multisampling support for all the DXGI_FORMAT values:

```
C++

// Determine the format support for multisampling.
for (UINT i = 1; i < DXGI_FORMAT_MAX; i++)
{
    DXGI_FORMAT inFormat = safe_cast<DXGI_FORMAT>(i);
    UINT formatSupport = 0;
    HRESULT hr = m_d3dDevice->CheckFormatSupport(inFormat,
    &formatSupport);

    if ((formatSupport & D3D11_FORMAT_SUPPORT_MULTISAMPLE_RESOLVE) &&
        (formatSupport & D3D11_FORMAT_SUPPORT_MULTISAMPLE_RENDER_TARGET))
    {
        m_supportInfo->SetFormatSupport(i, true);
    }
    else
    {
        m_supportInfo->SetFormatSupport(i, false);
    }
}
```

2. For each supported format, query for sample count support by calling [ID3D11Device::CheckMultisampleQualityLevels](#).

The following code checks sample size support for supported DXGI formats:

```
C++

// Find available sample sizes for each supported format.
for (unsigned int i = 0; i < DXGI_FORMAT_MAX; i++)
{
    for (unsigned int j = 1; j < MAX_SAMPLES_CHECK; j++)
    {
        UINT numQualityFlags;

        HRESULT test = m_d3dDevice->CheckMultisampleQualityLevels(
            (DXGI_FORMAT) i,
            j,
            &numQualityFlags
        );

        if (SUCCEEDED(test) && (numQualityFlags > 0))
        {
            m_supportInfo->SetSampleSize(i, j, 1);
            m_supportInfo->SetQualityFlagsAt(i, j, numQualityFlags);
        }
    }
}
```

Note Use [ID3D11Device2::CheckMultisampleQualityLevels1](#) instead if you need to check multisample support for tiled resource buffers.

3. Create a buffer and render target view with the desired sample count. Use the same DXGI_FORMAT, width, and height as the swap chain, but specify a sample count greater than 1 and use a multisampled texture dimension (**D3D11_RTV_DIMENSION_TEXTURE2DMS** for example). If necessary, you can re-create the swap chain with new settings that are optimal for multisampling.

The following code creates a multisampled render target:

C++

```
float widthMulti = m_d3dRenderTargetSize.Width;
float heightMulti = m_d3dRenderTargetSize.Height;

D3D11_TEXTURE2D_DESC offScreenSurfaceDesc;
ZeroMemory(&offScreenSurfaceDesc, sizeof(D3D11_TEXTURE2D_DESC));

offScreenSurfaceDesc.Format = DXGI_FORMAT_B8G8R8A8_UNORM;
offScreenSurfaceDesc.Width = static_cast<UINT>(widthMulti);
offScreenSurfaceDesc.Height = static_cast<UINT>(heightMulti);
offScreenSurfaceDesc.BindFlags = D3D11_BIND_RENDER_TARGET;
offScreenSurfaceDesc.MipLevels = 1;
offScreenSurfaceDesc.ArraySize = 1;
offScreenSurfaceDesc.SampleDesc.Count = m_sampleSize;
offScreenSurfaceDesc.SampleDesc.Quality = m_qualityFlags;

// Create a surface that's multisampled.
DX::ThrowIfFailed(
    m_d3dDevice->CreateTexture2D(
        &offScreenSurfaceDesc,
        nullptr,
        &m_offScreenSurface)
    );

// Create a render target view.
CD3D11_RENDER_TARGET_VIEW_DESC
renderTargetViewDesc(D3D11_RTV_DIMENSION_TEXTURE2DMS);
DX::ThrowIfFailed(
    m_d3dDevice->CreateRenderTargetView(
        m_offScreenSurface.Get(),
        &renderTargetViewDesc,
        &m_d3dRenderTargetView
    )
    );
```

4. The depth buffer must have the same width, height, sample count, and texture dimension to match the multisampled render target.

The following code creates a multisampled depth buffer:

C++

```
// Create a depth stencil view for use with 3D rendering if needed.
CD3D11_TEXTURE2D_DESC depthStencilDesc(
    DXGI_FORMAT_D24_UNORM_S8_UINT,
    static_cast<UINT>(widthMulti),
    static_cast<UINT>(heightMulti),
    1, // This depth stencil view has only one texture.
    1, // Use a single mipmap level.
    D3D11_BIND_DEPTH_STENCIL,
    D3D11_USAGE_DEFAULT,
    0,
    m_sampleSize,
    m_qualityFlags
);

ComPtr<ID3D11Texture2D> depthStencil;
DX::ThrowIfFailed(
    m_d3dDevice->CreateTexture2D(
        &depthStencilDesc,
        nullptr,
        &depthStencil
    )
);

CD3D11_DEPTH_STENCIL_VIEW_DESC
depthStencilViewDesc(D3D11_DSV_DIMENSION_TEXTURE2DMS);
DX::ThrowIfFailed(
    m_d3dDevice->CreateDepthStencilView(
        depthStencil.Get(),
        &depthStencilViewDesc,
        &m_d3dDepthStencilView
    )
);
```

5. Now is a good time to create the viewport, because the viewport width and height must also match the render target.

The following code creates a viewport:

C++

```
// Set the 3D rendering viewport to target the entire window.
m_screenViewport = CD3D11_VIEWPORT(
    0.0f,
    0.0f,
```

```

        widthMulti / m_scalingFactor,
        heightMulti / m_scalingFactor
    );

    m_d3dContext->RSSetViewports(1, &m_screenViewport);

```

6. Render each frame to the multisampled render target. When rendering is complete, call **ID3D11DeviceContext::ResolveSubresource** before presenting the frame. This instructs Direct3D to perform the multisampling operation, computing the value of each pixel for display and placing the result in the back buffer. The back buffer then contains the final anti-aliased image and can be presented.

The following code resolves the subresource before presenting the frame:

```

C++

if (m_sampleSize > 1)
{
    unsigned int sub = D3D11CalcSubresource(0, 0, 1);

    m_d3dContext->ResolveSubresource(
        m_backBuffer.Get(),
        sub,
        m_offScreenSurface.Get(),
        sub,
        DXGI_FORMAT_B8G8R8A8_UNORM
    );
}

// The first argument instructs DXGI to block until VSync, putting the
// application
// to sleep until the next VSync. This ensures that we don't waste any
// cycles rendering
// frames that will never be displayed to the screen.
hr = m_swapChain->Present(1, 0);

```

Optimize input latency for Universal Windows Platform (UWP) DirectX games

Article • 10/20/2022

Input latency can significantly impact the experience of a game, and optimizing it can make a game feel more polished. Additionally, proper input event optimization can improve battery life. Learn how to choose the right CoreDispatcher input event processing options to make sure your game handles input as smoothly as possible.

Input latency

Input latency is the time it takes for the system to respond to user input. The response is often a change in what's displayed on the screen, or what's heard through audio feedback.

Every input event, whether it comes from a touch pointer, mouse pointer, or keyboard, generates a message to be processed by an event handler. Modern touch digitizers and gaming peripherals report input events at a minimum of 100 Hz per pointer, which means that apps can receive 100 events or more per second per pointer (or keystroke). This rate of updates is amplified if multiple pointers are happening concurrently, or a higher precision input device is used (for example, a gaming mouse). The event message queue can fill up very quickly.

It's important to understand the input latency demands of your game so that events are processed in a way that is best for the scenario. There is no one solution for all games.

Power efficiency

In the context of input latency, "power efficiency" refers to how much a game uses the GPU. A game that uses less GPU resources is more power efficient and allows for longer battery life. This also holds true for the CPU.

If a game can draw the whole screen at less than 60 frames per second (currently, the maximum rendering speed on most displays) without degrading the user's experience, it will be more power efficient by drawing less often. Some games only update the screen in response to user input, so those games should not draw the same content repeatedly at 60 frames per second.

Choosing what to optimize for

When designing a DirectX app, you need to make some choices. Does the app need to render 60 frames per second to present smooth animation, or does it only need to render in response to input? Does it need to have the lowest possible input latency, or can it tolerate a little bit of delay? Will my users expect my app to be judicious about battery usage?

The answers to these questions will likely align your app with one of the following scenarios:

1. Render on demand. Games in this category only need to update the screen in response to specific types of input. Power efficiency is excellent because the app doesn't render identical frames repeatedly, and input latency is low because the app spends most of its time waiting for input. Board games and news readers are examples of apps that might fall into this category.
2. Render on demand with transient animations. This scenario is similar to the first scenario except that certain types of input will start an animation that isn't dependent on subsequent input from the user. Power efficiency is good because the game doesn't render identical frames repeatedly, and input latency is low while the game is not animating. Interactive children's games and board games that animate each move are examples of apps that might fall into this category.
3. Render 60 frames per second. In this scenario, the game is constantly updating the screen. Power efficiency is poor because it renders the maximum number of frames the display can present. Input latency is high because DirectX blocks the thread while content is being presented. Doing so prevents the thread from sending more frames to the display than it can show to the user. First person shooters, real-time strategy games, and physics-based games are examples of apps that might fall into this category.
4. Render 60 frames per second and achieve the lowest possible input latency. Similar to scenario 3, the app is constantly updating the screen, so power efficiency will be poor. The difference is that the game responds to input on a separate thread, so that input processing isn't blocked by presenting graphics to the display. Online multiplayer games, fighting games, or rhythm/timing games might fall into this category because they support move inputs within extremely tight event windows.

Implementation

Most DirectX games are driven by what is known as the game loop. The basic algorithm is to perform these steps until the user quits the game or app:

1. Process input
2. Update the game state
3. Draw the game content

When the content of a DirectX game is rendered and ready to be presented to the screen, the game loop waits until the GPU is ready to receive a new frame before waking up to process input again.

We'll show the implementation of the game loop for each of the scenarios mentioned earlier by iterating on a simple jigsaw puzzle game. The decision points, benefits, and tradeoffs discussed with each implementation can serve as a guide to help you optimize your apps for low latency input and power efficiency.

Scenario 1: Render on demand

The first iteration of the jigsaw puzzle game only updates the screen when a user moves a puzzle piece. A user can either drag a puzzle piece into place or snap it into place by selecting it and then touching the correct destination. In the second case, the puzzle piece will jump to the destination with no animation or effects.

The code has a single-threaded game loop within the [IFrameworkView::Run](#) method that uses [CoreProcessEventsOption::ProcessOneAndAllPending](#). Using this option dispatches all currently available events in the queue. If no events are pending, the game loop waits until one appears.

syntax

```
void App::Run()
{
    while (!m_windowClosed)
    {
        // Wait for system events or input from the user.
        // ProcessOneAndAllPending will block the thread until events appear
        and are processed.
        CoreWindow::GetForCurrentThread()->Dispatcher-
        >ProcessEvents(CoreProcessEventsOption::ProcessOneAndAllPending);

        // If any of the events processed resulted in a need to redraw the
        window contents, then we will re-render the
        // scene and present it to the display.
        if (m_updateWindow || m_state->StateChanged())
        {
            m_main->Render();
            m_deviceResources->Present();

            m_updateWindow = false;
        }
    }
}
```

```

        m_state->Validate();
    }
}
}

```

Scenario 2: Render on demand with transient animations

In the second iteration, the game is modified so that when a user selects a puzzle piece and then touches the correct destination for that piece, it animates across the screen until it reaches its destination.

As before, the code has a single-threaded game loop that uses **ProcessOneAndAllPending** to dispatch input events in the queue. The difference now is that during an animation, the loop changes to use **CoreProcessEventsOption::ProcessAllIfPresent** so that it doesn't wait for new input events. If no events are pending, **ProcessEvents** returns immediately and allows the app to present the next frame in the animation. When the animation is complete, the loop switches back to **ProcessOneAndAllPending** to limit screen updates.

syntax

```

void App::Run()
{
    while (!m_windowClosed)
    {
        // 2. Switch to a continuous rendering loop during the animation.
        if (m_state->Animating())
        {
            // Process any system events or input from the user that is
            // currently queued.
            // ProcessAllIfPresent will not block the thread to wait for
            // events. This is the desired behavior when
            // you are trying to present a smooth animation to the user.
            CoreWindow::GetForCurrentThread()->Dispatcher-
            >ProcessEvents(CoreProcessEventsOption::ProcessAllIfPresent);

            m_state->Update();
            m_main->Render();
            m_deviceResources->Present();
        }
        else
        {
            // Wait for system events or input from the user.
            // ProcessOneAndAllPending will block the thread until events
            // appear and are processed.
            CoreWindow::GetForCurrentThread()->Dispatcher-

```



```

>ProcessEvents(CoreProcessEventsOption::ProcessOneAndAllPending);

        // If any of the events processed resulted in a need to redraw
the window contents, then we will re-render the
        // scene and present it to the display.
        if (m_updateWindow || m_state->StateChanged())
        {
            m_main->Render();
            m_deviceResources->Present();

            m_updateWindow = false;
            m_state->Validate();
        }
    }
}

```

To support the transition between **ProcessOneAndAllPending** and **ProcessAllIfPresent**, the app must track state to know if it's animating. In the jigsaw puzzle app, you do this by adding a new method that can be called during the game loop on the `GameState` class. The animation branch of the game loop drives updates in the state of the animation by calling `GameState`'s new `Update` method.

Scenario 3: Render 60 frames per second

In the third iteration, the app displays a timer that shows the user how long they've been working on the puzzle. Because it displays the elapsed time up to the millisecond, it must render 60 frames per second to keep the display up to date.

As in scenarios 1 and 2, the app has a single-threaded game loop. The difference with this scenario is that because it's always rendering, it no longer needs to track changes in the game state as was done in the first two scenarios. As a result, it can default to use **ProcessAllIfPresent** for processing events. If no events are pending, **ProcessEvents** returns immediately and proceeds to render the next frame.

syntax

```

void App::Run()
{
    while (!m_windowClosed)
    {
        if (m_windowVisible)
        {
            // 3. Continuously render frames and process system events and
input as they appear in the queue.
            // ProcessAllIfPresent will not block the thread to wait for
events. This is the desired behavior when

```

```

        // trying to present smooth animations to the user.
        CoreWindow::GetForCurrentThread()->Dispatcher-
>ProcessEvents(CoreProcessEventsOption::ProcessAllIfPresent);

        m_state->Update();
        m_main->Render();
        m_deviceResources->Present();
    }
    else
    {
        // 3. If the window isn't visible, there is no need to
        continuously render.
        // Process events as they appear until the window becomes
        visible again.
        CoreWindow::GetForCurrentThread()->Dispatcher-
>ProcessEvents(CoreProcessEventsOption::ProcessOneAndAllPending);
    }
}
}
}

```

This approach is the easiest way to write a game because there's no need to track additional state to determine when to render. It achieves the fastest rendering possible along with reasonable input responsiveness on a timer interval.

However, this ease of development comes with a price. Rendering at 60 frames per second uses more power than rendering on demand. It's best to use **ProcessAllIfPresent** when the game is changing what is displayed every frame. It also increases input latency by as much as 16.7 ms because the app is now blocking the game loop on the display's sync interval instead of on **ProcessEvents**. Some input events might be dropped because the queue is only processed once per frame (60 Hz).

Scenario 4: Render 60 frames per second and achieve the lowest possible input latency

Some games may be able to ignore or compensate for the increase in input latency seen in scenario 3. However, if low input latency is critical to the game's experience and sense of player feedback, games that render 60 frames per second need to process input on a separate thread.

The fourth iteration of the jigsaw puzzle game builds on scenario 3 by splitting the input processing and graphics rendering from the game loop into separate threads. Having separate threads for each ensures that input is never delayed by graphics output; however, the code becomes more complex as a result. In scenario 4, the input thread calls **ProcessEvents** with **CoreProcessEventsOption::ProcessUntilQuit**, which waits for

new events and dispatches all available events. It continues this behavior until the window is closed or the game calls [CoreWindow::Close](#).

syntax

```
void App::Run()
{
    // 4. Start a thread dedicated to rendering and dedicate the UI thread
    to input processing.
    m_main->StartRenderThread();

    // ProcessUntilQuit will block the thread and process events as they
    appear until the App terminates.
    CoreWindow::GetForCurrentThread()->Dispatcher-
    >ProcessEvents(CoreProcessEventsOption::ProcessUntilQuit);
}

void JigsawPuzzleMain::StartRenderThread()
{
    // If the render thread is already running, then do not start another
    one.
    if (IsRendering())
    {
        return;
    }

    // Create a task that will be run on a background thread.
    auto workItemHandler = ref new WorkItemHandler([this](IAsyncAction^
    action)
    {
        // Notify the swap chain that this app intends to render each frame
        faster
        // than the display's vertical refresh rate (typically 60 Hz). Apps
        that cannot
        // deliver frames this quickly should set this to 2.
        m_deviceResources->SetMaximumFrameLatency(1);

        // Calculate the updated frame and render once per vertical blanking
        interval.
        while (action->Status == AsyncStatus::Started)
        {
            // Execute any work items that have been queued by the input
            thread.
            ProcessPendingWork();

            // Take a snapshot of the current game state. This allows the
            renderers to work with a
            // set of values that won't be changed while the input thread
            continues to process events.
            m_state->SnapState();

            m_sceneRenderer->Render();
            m_deviceResources->Present();
        }
    })
}
```

```
// Ensure that all pending work items have been processed before
terminating the thread.
    ProcessPendingWork();
});

// Run the task on a dedicated high priority background thread.
m_renderLoopWorker = ThreadPool::RunAsync(workItemHandler,
WorkItemPriority::High, WorkItemOptions::TimeSliced);
}
```

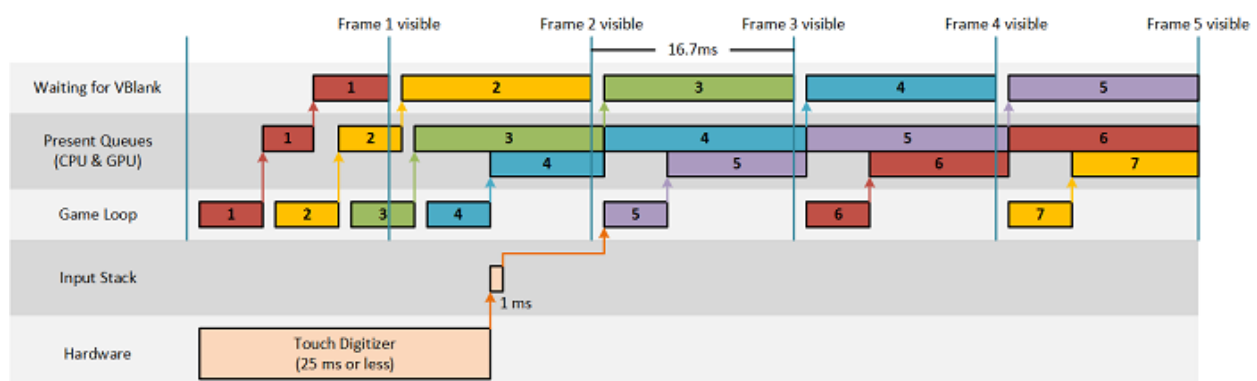
The **DirectX 11 and XAML App (Universal Windows)** template in Microsoft Visual Studio 2015 splits the game loop into multiple threads in a similar fashion. It uses the [Windows::UI::Core::CoreIndependentInputSource](#) object to start a thread dedicated to handling input and also creates a rendering thread independent of the XAML UI thread. For more details on these templates, read [Create a Universal Windows Platform and DirectX game project from a template](#).

Additional ways to reduce input latency

Use waitable swap chains

DirectX games respond to user input by updating what the user sees on-screen. On a 60 Hz display, the screen refreshes every 16.7 ms (1 second/60 frames). Figure 1 shows the approximate life cycle and response to an input event relative to the 16.7 ms refresh signal (VBlank) for an app that renders 60 frames per second:

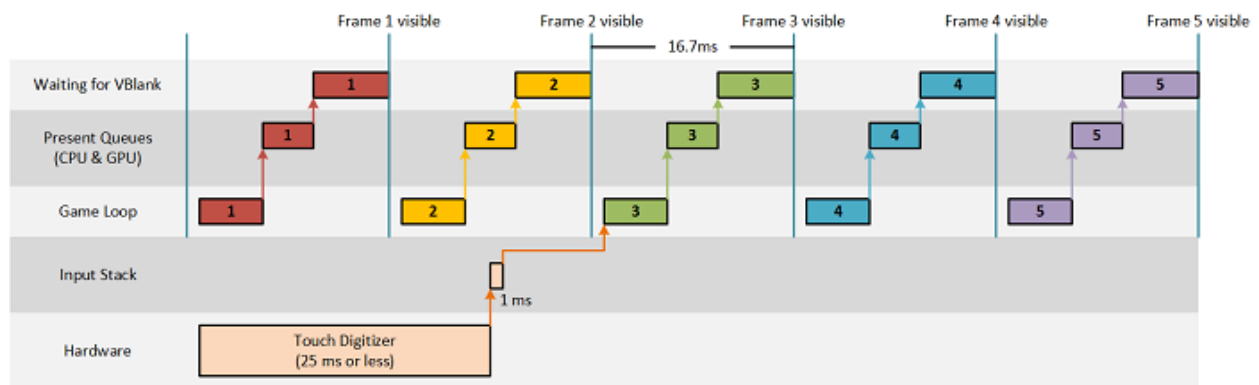
Figure 1



In Windows 8.1, DXGI introduced the **DXGI_SWAP_CHAIN_FLAG_FRAME_LATENCY_WAITABLE_OBJECT** flag for the swap chain, which allows apps to easily reduce this latency without requiring them to implement heuristics to keep the Present queue empty. Swap chains created with this

flag are referred to as waitable swap chains. Figure 2 shows the approximate life cycle and response to an input event when using waitable swap chains:

Figure 2



What we see from these diagrams is that games can potentially reduce input latency by two full frames if they are capable of rendering and presenting each frame within the 16.7 ms budget defined by the display's refresh rate. The jigsaw puzzle sample uses waitable swap chains and controls the Present queue limit by calling:

```
m_deviceResources->SetMaximumFrameLatency(1);
```

Reduce latency with DXGI 1.3 swap chains

Article • 10/20/2022

Use DXGI 1.3 to reduce the effective frame latency by waiting for the swap chain to signal the appropriate time to begin rendering a new frame. Games typically need to provide the lowest amount of latency possible from the time the player input is received, to when the game responds to that input by updating the display. This topic explains a technique available starting in Direct3D 11.2 that you can use to minimize the effective frame latency in your game.

How does waiting on the back buffer reduce latency?

With the flip model swap chain, back buffer "flips" are queued whenever your game calls [IDXGISwapChain::Present](#). When the rendering loop calls `Present()`, the system blocks the thread until it is done presenting a prior frame, making room to queue up the new frame, before it actually presents. This causes extra latency between the time the game draws a frame and the time the system allows it to display that frame. In many cases, the system will reach a stable equilibrium where the game is always waiting almost a full extra frame between the time it renders and the time it presents each frame. It's better to wait until the system is ready to accept a new frame, then render the frame based on current data and queue the frame immediately.

Create a waitable swap chain with the [DXGI_SWAP_CHAIN_FLAG_FRAME_LATENCY_WAITABLE_OBJECT](#) flag. Swap chains created this way can notify your rendering loop when the system is actually ready to accept a new frame. This allows your game to render based on current data and then put the result in the present queue right away.

Step 1. Create a waitable swap chain

Specify the [DXGI_SWAP_CHAIN_FLAG_FRAME_LATENCY_WAITABLE_OBJECT](#) flag when you call [CreateSwapChainForCoreWindow](#).

C++

```
swapChainDesc.Flags = DXGI_SWAP_CHAIN_FLAG_FRAME_LATENCY_WAITABLE_OBJECT; //  
Enable GetFrameLatencyWaitableObject().
```

❗ Note

In contrast to some flags, this flag can't be added or removed using **ResizeBuffers**. DXGI returns an error code if this flag is set differently from when the swap chain was created.

C++

```
// If the swap chain already exists, resize it.
HRESULT hr = m_swapChain->ResizeBuffers(
    2, // Double-buffered swap chain.
    static_cast<UINT>(m_d3dRenderTargetSize.Width),
    static_cast<UINT>(m_d3dRenderTargetSize.Height),
    DXGI_FORMAT_B8G8R8A8_UNORM,
    DXGI_SWAP_CHAIN_FLAG_FRAME_LATENCY_WAITABLE_OBJECT // Enable
    GetFrameLatencyWaitableObject().
);
```

Step 2. Set the frame latency

Set the frame latency with the **IDXGISwapChain2::SetMaximumFrameLatency** API, instead of calling **IDXGIDevice1::SetMaximumFrameLatency**.

By default, the frame latency for waitable swap chains is set to 1, which results in the least possible latency but also reduces CPU-GPU parallelism. If you need increased CPU-GPU parallelism to achieve 60 FPS - that is, if the CPU and GPU each spend less than 16.7 ms a frame processing rendering work, but their combined sum is greater than 16.7 ms — set the frame latency to 2. This allows the GPU to process work queued up by the CPU during the previous frame, while at the same time allowing the CPU to submit rendering commands for the current frame independently.

C++

```
// Swapchains created with the
DXGI_SWAP_CHAIN_FLAG_FRAME_LATENCY_WAITABLE_OBJECT flag use their
// own per-swapchain latency setting instead of the one associated with the
DXGI device. The
// default per-swapchain latency is 1, which ensures that DXGI does not
queue more than one frame
// at a time. This both reduces latency and ensures that the application
will only render after
// each VSync, minimizing power consumption.
//DX::ThrowIfFailed(
```

```
// swapChain2->SetMaximumFrameLatency(1)
// );
```

Step 3. Get the waitable object from the swap chain

Call [IDXGISwapChain2::GetFrameLatencyWaitableObject](#) to retrieve the wait handle. The wait handle is a pointer to the waitable object. Store this handle for use by your rendering loop.

C++

```
// Get the frame latency waitable object, which is used by the
WaitOnSwapChain method. This
// requires that swap chain be created with the
DXGI_SWAP_CHAIN_FLAG_FRAME_LATENCY_WAITABLE_OBJECT
// flag.
m_frameLatencyWaitableObject = swapChain2->GetFrameLatencyWaitableObject();
```

Step 4. Wait before rendering each frame

Your rendering loop should wait for the swap chain to signal via the waitable object before it begins rendering every frame. This includes the first frame rendered with the swap chain. Use [WaitForSingleObjectEx](#), providing the wait handle retrieved in Step 2, to signal the start of each frame.

The following example shows the render loop from the DirectXLatency sample:

C++

```
while (!m_windowClosed)
{
    if (m_windowVisible)
    {
        // Block this thread until the swap chain is finished presenting.
        Note that it is
        // important to call this before the first Present in order to
        minimize the latency
        // of the swap chain.
        m_deviceResources->WaitOnSwapChain();

        // Process any UI events in the queue.
        CoreWindow::GetForCurrentThread()->Dispatcher-
        >ProcessEvents(CoreProcessEventsOption::ProcessAllIfPresent);
```



```

        // Update app state in response to any UI events that occurred.
        m_main->Update();

        // Render the scene.
        m_main->Render();

        // Present the scene.
        m_deviceResources->Present();
    }
    else
    {
        // The window is hidden. Block until a UI event occurs.
        CoreWindow::GetForCurrentThread()->Dispatcher-
        >ProcessEvents(CoreProcessEventsOption::ProcessOneAndAllPending);
    }
}

```

The following example shows the `WaitForSingleObjectEx` call from the `DirectXLatency` sample:

C++

```

// Block the current thread until the swap chain has finished presenting.
void DX::DeviceResources::WaitOnSwapChain()
{
    DWORD result = WaitForSingleObjectEx(
        m_frameLatencyWaitableObject,
        1000, // 1 second timeout (shouldn't ever occur)
        true
    );
}

```

What should my game do while it waits for the swap chain to present?

If your game doesn't have any tasks that block on the render loop, letting it wait for the swap chain to present can be advantageous because it saves power, which is especially important on mobile devices. Otherwise, you can use multithreading to accomplish work while your game is waiting for the swap chain to present. Here are just a few tasks that your game can complete:

- Process network events
- Update the AI
- CPU-based physics
- Deferred-context rendering (on supported devices)
- Asset loading

For more information about multithreaded programming in Windows, see the following related topics.

Related topics

- [DirectX latency sample application](#) 
- [IDXGISwapChain2::GetFrameLatencyWaitableObject](#)
- [WaitForSingleObjectEx](#)
- [Windows.System.Threading](#)
- [Asynchronous programming in C++](#)
- [Processes and Threads](#)
- [Synchronization](#)
- [Using Event Objects \(Windows\)](#)

Swap chain scaling and overlays

Article • 10/20/2022

Learn how to create scaled swap chains for faster rendering on mobile devices, and use overlay swap chains (when available) to increase the visual quality.

Swap chains in DirectX 11.2

Direct3D 11.2 allows you to create Universal Windows Platform (UWP) apps with swap chains that are scaled up from non-native (reduced) resolutions, enabling faster fill rates. Direct3D 11.2 also includes APIs for rendering with hardware overlays so that you can present a UI in another swap chain at native resolution. This allows your game to draw UI at full native resolution while maintaining a high framerate, thereby making the best use of mobile devices and high DPI displays (such as 3840 by 2160). This article explains how to use overlapping swap chains.

Direct3D 11.2 also introduces a new feature for reduced latency with flip model swap chains. See [Reduce latency with DXGI 1.3 swap chains](#).

Use swap chain scaling

When your game is running on downlevel hardware - or hardware optimized for power savings - it can be beneficial to render real-time game content at a lower resolution than the display is natively capable of. To do this, the swap chain that is used for rendering game content must be smaller than the native resolution, or a subregion of the swapchain must be used.

1. First, create a swap chain at full native resolution.

C++

```
DXGI_SWAP_CHAIN_DESC1 swapChainDesc = {0};

swapChainDesc.Width = static_cast<UINT>(m_d3dRenderTargetSize.Width);
// Match the size of the window.
swapChainDesc.Height = static_cast<UINT>(m_d3dRenderTargetSize.Height);
swapChainDesc.Format = DXGI_FORMAT_B8G8R8A8_UNORM; // This is the most
common swap chain format.
swapChainDesc.Stereo = false;
swapChainDesc.SampleDesc.Count = 1; // Don't use multi-sampling.
swapChainDesc.SampleDesc.Quality = 0;
swapChainDesc.BufferUsage = DXGI_USAGE_RENDER_TARGET_OUTPUT;
swapChainDesc.BufferCount = 2; // Use double-buffering to minimize
```

```

latency.
swapChainDesc.SwapEffect = DXGI_SWAP_EFFECT_FLIP_SEQUENTIAL; // All UWP
apps must use this SwapEffect.
swapChainDesc.Flags = 0;
swapChainDesc.Scoring = DXGI_SCALING_STRETCH;

// This sequence obtains the DXGI factory that was used to create the
Direct3D device above.
ComPtr<IDXGIDevice3> dxgiDevice;
DX::ThrowIfFailed(
    m_d3dDevice.As(&dxgiDevice)
);

ComPtr<IDXGIAdapter> dxgiAdapter;
DX::ThrowIfFailed(
    dxgiDevice->GetAdapter(&dxgiAdapter)
);

ComPtr<IDXGIFactory2> dxgiFactory;
DX::ThrowIfFailed(
    dxgiAdapter->GetParent(IID_PPV_ARGS(&dxgiFactory))
);

ComPtr<IDXGISwapChain1> swapChain;
DX::ThrowIfFailed(
    dxgiFactory->CreateSwapChainForCoreWindow(
        m_d3dDevice.Get(),
        reinterpret_cast<IUnknown*>(m_window.Get()),
        &swapChainDesc,
        nullptr,
        &swapChain
    )
);

DX::ThrowIfFailed(
    swapChain.As(&m_swapChain)
);

```

2. Then, choose a subregion of the swap chain to scale up by setting the source size to a reduced resolution.

The DX Foreground Swap Chains sample calculates a reduced size based on a percentage:

C++

```

m_d3dRenderSizePercentage = percentage;

UINT renderWidth = static_cast<UINT>(m_d3dRenderTargetSize.Width *
percentage + 0.5f);
UINT renderHeight = static_cast<UINT>(m_d3dRenderTargetSize.Height *
percentage + 0.5f);

```

```
// Change the region of the swap chain that will be presented to the
screen.
DX::ThrowIfFailed(
    m_swapChain->SetSourceSize(
        renderWidth,
        renderHeight
    )
);
```

3. Create a viewport to match the subregion of the swap chain.

```
C++

// In Direct3D, change the Viewport to match the region of the swap
// chain that will now be presented from.
m_screenViewport = CD3D11_VIEWPORT(
    0.0f,
    0.0f,
    static_cast<float>(renderWidth),
    static_cast<float>(renderHeight)
);

m_d3dContext->RSSetViewports(1, &m_screenViewport);
```

4. If Direct2D is being used, the rotation transform needs to be adjusted to compensate for the source region.

Create a hardware overlay swap chain for UI elements

When using swap chain scaling, there is an inherent disadvantage in that the UI is also scaled down, potentially making it blurry and harder to use. On devices with hardware support for overlay swap chains, this problem is alleviated entirely by rendering the UI at native resolution in a swap chain that's separate from the real-time game content. Note that this technique applies only to [CoreWindow](#) swap chains - it cannot be used with XAML interop.

Use the following steps to create a foreground swap chain that uses hardware overlay capability. These steps are performed after first creating a swap chain for real-time game content as described above.

1. First, determine whether the DXGI adapter supports overlays. Get the DXGI output adapter from the swap chain: